

CART in R

Kachun Huang

2025-03-25

CART in R

Import Dependencies

```
library(rpart)
```

```
## Warning: package 'rpart' was built under R version 4.3.3
```

```
library(rpart.plot)
```

```
## Warning: package 'rpart.plot' was built under R version 4.3.3
```

```
library(rattle)
```

```
## Loading required package: tibble
```

```
## Loading required package: bitops
```

```
## Warning: package 'bitops' was built under R version 4.3.3
```

```
## Rattle: A free graphical interface for data science with R.  
## Version 5.5.1 Copyright (c) 2006-2021 Togaware Pty Ltd.  
## Type 'rattle()' to shake, rattle, and roll your data.
```

```
library(caTools)
```

```
## Warning: package 'caTools' was built under R version 4.3.3
```

```
library(tidyverse)
```

```
## Warning: package 'purrr' was built under R version 4.3.3
```

```
## Warning: package 'lubridate' was built under R version 4.3.3
```

```
## -- Attaching core tidyverse packages ----- tidyverse 2.0.0 --
## v dplyr      1.1.4      v purrr      1.0.4
## v forcats    1.0.0      v readr      2.1.5
## v ggplot2    3.5.1      v stringr   1.5.1
## v lubridate  1.9.4      v tidyr      1.3.1

## -- Conflicts ----- tidyverse_conflicts() --
## x dplyr::filter() masks stats::filter()
## x dplyr::lag()     masks stats::lag()
## i Use the conflicted package (<http://conflicted.r-lib.org/>) to force all conflicts to become errors
```

```
library(caret)
```

```
## Warning: package 'caret' was built under R version 4.3.3
```

```
## Loading required package: lattice
```

```
## Warning: package 'lattice' was built under R version 4.3.3
```

```
##
## Attaching package: 'caret'
##
## The following object is masked from 'package:purrr':
##
##     lift
```

Model Fitting

syntax:

```
model <- rpart(
  Y~.,
  data,
  method,
  control = rpart.control(cp=...)
)
```

parameters:

- Method: one of "anova", "poisson", "class" or "exp". If **method** is missing then the routine tries to make an intelligent guess. If **y** is a survival object, then **method** = "exp" is assumed, if **y** has 2 columns then **method** = "poisson" is assumed, if **y** is a factor then **method** = "class" is assumed, otherwise **method** = "anova" is assumed. It is wisest to specify the method directly, especially as more criteria may be added to the function in future.
- control: a list of options that control details of the **rpart** algorithm.
 - in **rpart.control**, **cp** is the complexity parameter.
 - * **cp** controls the trade-off between tree size and accuracy
 - * when **cp** = 0, creates a fully grown tree. more splits, more details but overfitting

Fit Controlled Model

To avoid overfitting, we can set constraints like `minsplit` and `minbucket`.

- `minsplit`: Minimum number of observations required to split a node.
- `minbucket`: Minimum number of observations in a leaf node.

```
model <- rpart(  
  Y~.,  
  data,  
  method,  
  control = rpart.control(cp=..., minsplit=..., minbucket =...)  
)
```

Fit a Tree Using Entropy

By default, `rpart` uses the Gini index for splits. To use entropy instead, specify the `split` parameter.

```
model <- rpart(  
  Y~.,  
  data,  
  method,  
  params = list(split="information") #information gain (entropy)  
)
```

Model Plot

This is from `rattle` packages

```
# Visualize the tree using fancyRpartPlot from rattle  
par(xpd = NA) # Allow plotting outside the margins  
fancyRpartPlot(model,  
  sub = "",  
  caption = "",  
  main = "Title_name",  
  split.col = "blue", # Color of split text  
  box.palette = "RdYlBu", # Color scheme  
  tweak = 1.2 # Adjust node size  
)
```

or we can use `rpart.plot` package

```
rpart.plot(  
  model,  
  type = 4, # Shows split labels at nodes  
  extra = 104, #Displays probabilities and class labels at terminal nodes.  
  tweak = 1.2 # Adjusts font size  
)
```

Prune the tree

Pruning reduces overfitting by selecting an optimal complexity parameter (`cp`) using cross-validation.

```
set.seed(123)
model_pruned <- train(
  Y~.,
  data,
  method = "rpart",
  trControl = trainControl("cv", number = 10), # 10 fold CV
  tuneLength = 100 # Tries 100 different complexity parameters (cp) to find the best one.
)
```

To see the Accuracy vs. `cp` plot

```
plot(model_pruned)
```

To access the best `cp`

```
model_pruned$bestTune
```

Prediction

Make predictions on the test set and evaluate performance using a confusion matrix.

```
# Predict on test data
pred <- predict(model, test.data, type = "class")

# confusion matrix (caret package)
confusionMatrix(factor(pred), factor(test.data$Response), positive = "1")
```

Example

Import Data

```
data <- read.csv("/Users/thomashuang/Downloads/Titanic.csv")
head(data)
```

```
##   PassengerId Survived Pclass
## 1           1         0       3
## 2           2         1       1
## 3           3         1       3
## 4           4         1       1
## 5           5         0       3
## 6           6         0       3
##                                     Name    Sex Age SibSp Parch
## 1                                Braund, Mr. Owen Harris  male  22     1     0
## 2 Cumings, Mrs. John Bradley (Florence Briggs Thayer) female  38     1     0
```

```
## 3                      Heikkinen, Miss. Laina female 26      0      0
## 4      Futrelle, Mrs. Jacques Heath (Lily May Peel) female 35      1      0
## 5                      Allen, Mr. William Henry   male 35      0      0
## 6                      Moran, Mr. James          male NA      0      0
##      Ticket      Fare Cabin Embarked
## 1      A/5 21171  7.2500      S
## 2      PC 17599 71.2833  C85      C
## 3 STON/O2. 3101282 7.9250      S
## 4      113803 53.1000  C123      S
## 5      373450 8.0500      S
## 6      330877 8.4583      Q
```

```
str(data)
```

```
## 'data.frame':      891 obs. of  12 variables:
## $ PassengerId: int  1 2 3 4 5 6 7 8 9 10 ...
## $ Survived   : int  0 1 1 1 0 0 0 0 1 1 ...
## $ Pclass     : int  3 1 3 1 3 3 1 3 3 2 ...
## $ Name       : chr  "Braund, Mr. Owen Harris" "Cumings, Mrs. John Bradley (Florence Briggs Thayer)"
## $ Sex        : chr  "male" "female" "female" "female" ...
## $ Age        : num  22 38 26 35 35 NA 54 2 27 14 ...
## $ SibSp      : int  1 1 0 1 0 0 0 3 0 1 ...
## $ Parch      : int  0 0 0 0 0 0 0 1 2 0 ...
## $ Ticket     : chr  "A/5 21171" "PC 17599" "STON/O2. 3101282" "113803" ...
## $ Fare       : num  7.25 71.28 7.92 53.1 8.05 ...
## $ Cabin      : chr  "" "C85" "" "C123" ...
## $ Embarked   : chr  "S" "C" "S" "S" ...
```

Data Pre-processing

Remove Missing Values

```
data <- na.omit(data)
```

```
str(data)
```

```
## 'data.frame':      714 obs. of  12 variables:
## $ PassengerId: int  1 2 3 4 5 7 8 9 10 11 ...
## $ Survived   : int  0 1 1 1 0 0 0 1 1 1 ...
## $ Pclass     : int  3 1 3 1 3 1 3 3 2 3 ...
## $ Name       : chr  "Braund, Mr. Owen Harris" "Cumings, Mrs. John Bradley (Florence Briggs Thayer)"
## $ Sex        : chr  "male" "female" "female" "female" ...
## $ Age        : num  22 38 26 35 35 54 2 27 14 4 ...
## $ SibSp      : int  1 1 0 1 0 0 3 0 1 1 ...
## $ Parch      : int  0 0 0 0 0 0 1 2 0 1 ...
## $ Ticket     : chr  "A/5 21171" "PC 17599" "STON/O2. 3101282" "113803" ...
## $ Fare       : num  7.25 71.28 7.92 53.1 8.05 ...
## $ Cabin      : chr  "" "C85" "" "C123" ...
## $ Embarked   : chr  "S" "C" "S" "S" ...
## - attr(*, "na.action")= 'omit' Named int [1:177] 6 18 20 27 29 30 32 33 37 43 ...
## ..- attr(*, "names")= chr [1:177] "6" "18" "20" "27" ...
```

Factorized categorical variable

```
data$Pclass <- factor(data$Pclass)
data$Survived <- factor(data$Survived)
```

Select only subset of the features

```
data <- subset(data, select=-c(Name, Ticket, Cabin))
str(data)
```

```
## 'data.frame': 714 obs. of 9 variables:
## $ PassengerId: int 1 2 3 4 5 7 8 9 10 11 ...
## $ Survived : Factor w/ 2 levels "0","1": 1 2 2 2 1 1 1 2 2 2 ...
## $ Pclass : Factor w/ 3 levels "1","2","3": 3 1 3 1 3 1 3 3 2 3 ...
## $ Sex : chr "male" "female" "female" "female" ...
## $ Age : num 22 38 26 35 35 54 2 27 14 4 ...
## $ SibSp : int 1 1 0 1 0 0 3 0 1 1 ...
## $ Parch : int 0 0 0 0 0 0 1 2 0 1 ...
## $ Fare : num 7.25 71.28 7.92 53.1 8.05 ...
## $ Embarked : chr "S" "C" "S" "S" ...
```

Data Splitting

```
set.seed(123)
training.sample <- data$Survived %>% createDataPartition(p=0.8, list = FALSE)
train.data <- data[training.sample,]
test.data <- data[-training.sample,]
```

Fit Full Tree Model

```
full <- rpart(
  Survived~.,
  data=train.data,
  control = rpart.control(cp=0) # fully grown tree
)
```

Plot

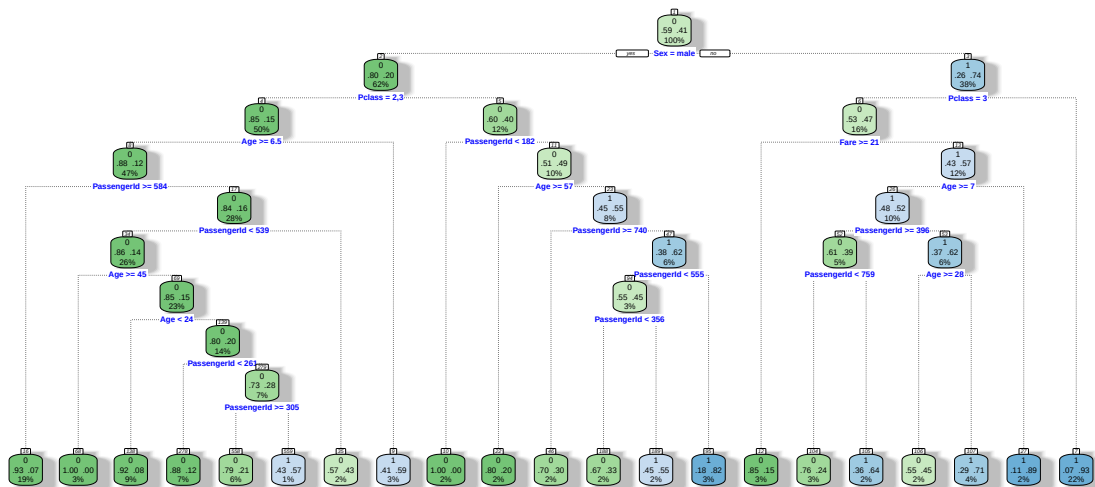
```
fancyRpartPlot(
  full,
  sub = "",
  caption = "",
  main = "Decision Tree for Survived",
  split.col = "blue", # Color of split text
)
```

```

box.palette = "RdYlBu", # Color scheme
tweak = 1.2 # Adjust node size
)

```

Decision Tree for Survived



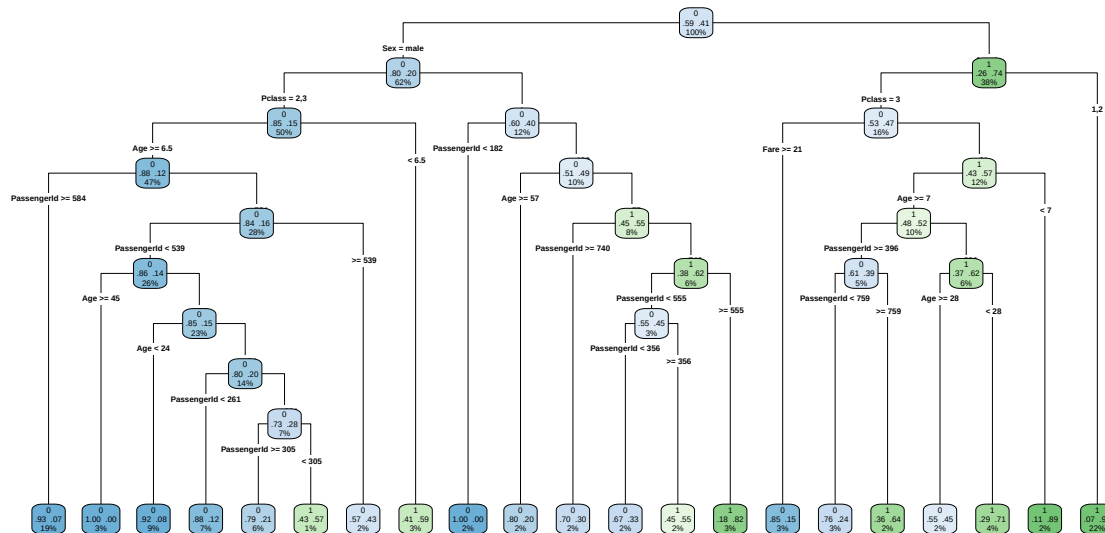
or use `rpart.plot` pkg

```

rpart.plot(
  full,
  type = 4, # type = 4: Shows split labels at nodes
  extra = 104, # extra = 104: Displays probabilities and class labels at terminal nodes
  tweak = 1.2, # tweak = 1.2: Adjusts font size
  main = "Decision Tree for Survived"
)

```

Decision Tree for Survived



Prediction

```
pred_full <- predict(
  full,
  test.data,
  type="class"
)
```

Confusion Matrix

```
confusionMatrix(
  factor(pred_full),
  factor(test.data$Survived),
  positive = "1"
)
```

```
## Confusion Matrix and Statistics
##
##           Reference
## Prediction  0  1
##           0 71 11
##           1 13 47
##
##           Accuracy : 0.831
##           95% CI : (0.759, 0.8886)
##           No Information Rate : 0.5915
##           P-Value [Acc > NIR] : 7.518e-10
```

```
##
##           Kappa : 0.6521
##
## Mcnemar's Test P-Value : 0.8383
##
##           Sensitivity : 0.8103
##           Specificity : 0.8452
##           Pos Pred Value : 0.7833
##           Neg Pred Value : 0.8659
##           Prevalence : 0.4085
##           Detection Rate : 0.3310
##           Detection Prevalence : 0.4225
##           Balanced Accuracy : 0.8278
##
##           'Positive' Class : 1
##
```

Fit Full Tree with Control

```
control_model <- rpart(
  Survived~.,
  data = train.data,
  control = rpart.control(cp=0,
                           minisplit=10,
                           minbucket=5)
)
```

Plot

```
fancyRpartPlot(
  control_model,
  sub = "",
  caption = "",
  main = "Decision Tree for Survived",
  split.col = "blue", # Color of split text
  box.palette = "RdYlBu", # Color scheme
  tweak = 1.2 # Adjust node size
)
```



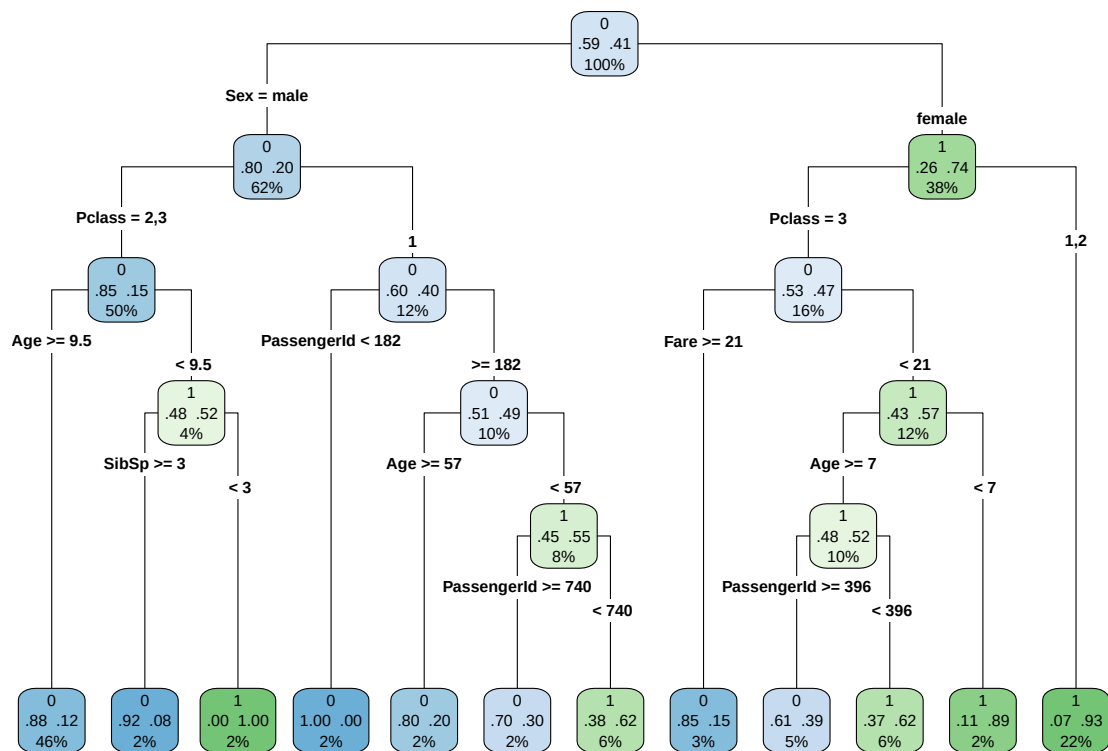
```
##      No Information Rate : 0.5915
##      P-Value [Acc > NIR] : 5.866e-11
##
##              Kappa : 0.6724
##
##  McNemar's Test P-Value : 0.1356
##
##      Sensitivity : 0.7414
##      Specificity : 0.9167
##      Pos Pred Value : 0.8600
##      Neg Pred Value : 0.8370
##      Prevalence : 0.4085
##      Detection Rate : 0.3028
##      Detection Prevalence : 0.3521
##      Balanced Accuracy : 0.8290
##
##      'Positive' Class : 1
##
```

Fit Tree with (Entropy)

```
model_entropy <- rpart(
  Survived ~ .,
  data = train.data,
  method = "class", # if resposne is factor
  parms = list(split = "information") # Use entropy
)
```

Plot

```
rpart.plot(
  model_entropy,
  type = 4,
  extra = 104,
  tweak = 1.2
)
```



Prediction

```

pred_entropy <- predict(
  model_entropy,
  test.data,
  type="class"
)

```

Confusion Matrix

```

confusionMatrix(
  factor(pred_entropy),
  factor(test.data$Survived),
  positive = "1"
)

```

```

## Confusion Matrix and Statistics
##
##           Reference
## Prediction  0    1
##           0  71   9
##           1  13  49
##
##               Accuracy : 0.8451
##               95% CI : (0.7749, 0.9003)

```

```
##      No Information Rate : 0.5915
##      P-Value [Acc > NIR] : 5.866e-11
##
##              Kappa : 0.6828
##
##  Mcnemar's Test P-Value : 0.5224
##
##      Sensitivity : 0.8448
##      Specificity : 0.8452
##      Pos Pred Value : 0.7903
##      Neg Pred Value : 0.8875
##      Prevalence : 0.4085
##      Detection Rate : 0.3451
##      Detection Prevalence : 0.4366
##      Balanced Accuracy : 0.8450
##
##      'Positive' Class : 1
##
```

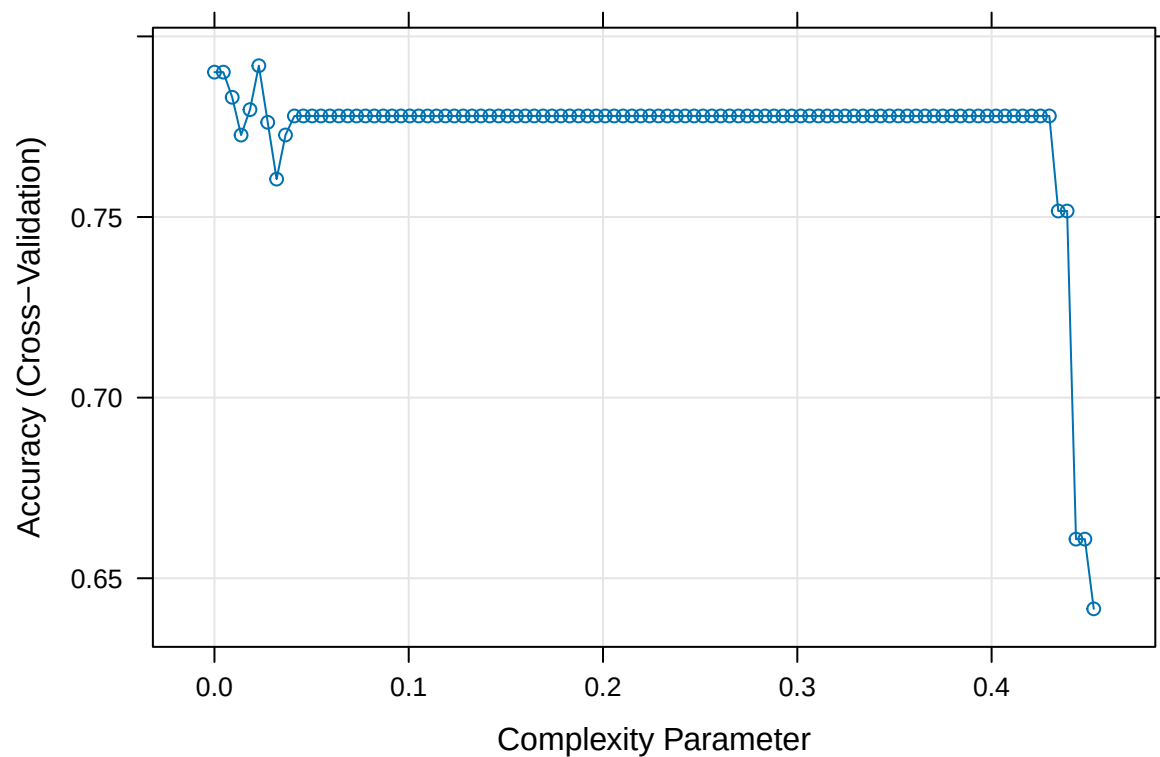
Pruning full tree with 10-fold CV

```
set.seed(123)

model2 <- train(
  Survived ~.,
  data = train.data,
  method = "rpart",
  trControl = trainControl("cv", number = 10), #trControl = trainControl("cv", number = 10): Implements
  tuneLength = 100 # tuneLength = 100: Tries 100 different complexity parameters (cp) to find the best
)
```

Plot

```
plot(model2)
```



The tree starts from its fully grown version, and then cross-validation selects the optimal cp value,

- 1. The tree is initially grown fully with different cp values.
- 2. 10-fold cross-validation evaluates tree performance for each cp.
- 3. The best cp (minimizing validation error) is selected.
- 4. The final tree is pruned using the best cp.

cp stands for Complexity Parameter:

- The best cp is the one that minimizes the cross-validation error

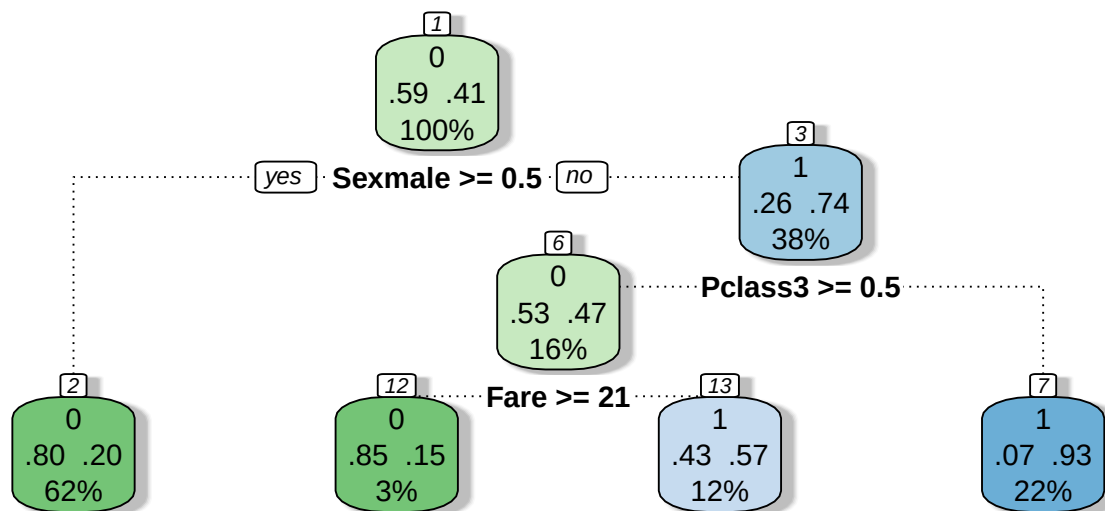
Access the best tune

```
model2$bestTune
```

```
##          cp
## 6 0.02285789
```

Plot the best model

```
fancyRpartPlot(model2$finalModel)
```



Rattle 2025-Mar-30 23:29:35 thomashuang

Prediction

```
pred_2 <- predict(
  model2,
  test.data
)
```

Confusion Matrix

```
confusionMatrix(
  factor(pred_2),
  factor(test.data$Survived),
  positive = "1"
)
```

```
## Confusion Matrix and Statistics
##
##           Reference
## Prediction  0  1
##           0 79 22
##           1  5 36
##
##           Accuracy : 0.8099
##           95% CI : (0.7355, 0.8708)
##           No Information Rate : 0.5915
##           P-Value [Acc > NIR] : 2.405e-08
##
##           Kappa : 0.5878
```

```
##
## McNemar's Test P-Value : 0.002076
##
##      Sensitivity : 0.6207
##      Specificity : 0.9405
##      Pos Pred Value : 0.8780
##      Neg Pred Value : 0.7822
##      Prevalence : 0.4085
##      Detection Rate : 0.2535
##      Detection Prevalence : 0.2887
##      Balanced Accuracy : 0.7806
##
##      'Positive' Class : 1
##
```

Manually Prune the Tree with rpart

```
full_tree <- rpart(Survived ~ ., data = train.data, cp = 0) # Fully grown tree
printcp(full_tree) # Check complexity parameters
```

```
##
## Classification tree:
## rpart(formula = Survived ~ ., data = train.data, cp = 0)
##
## Variables actually used in tree construction:
## [1] Age      Fare      PassengerId Pclass      Sex
##
## Root node error: 232/572 = 0.40559
##
## n= 572
##
##      CP nsplit rel error  xerror    xstd
## 1 0.45258621      0  1.00000 1.00000 0.050617
## 2 0.03017241      1  0.54741 0.54741 0.042845
## 3 0.01293103      3  0.48707 0.51724 0.041973
## 4 0.00862069      6  0.44828 0.55603 0.043083
## 5 0.00431034     12  0.38793 0.56034 0.043201
## 6 0.00071839     14  0.37931 0.56466 0.043318
## 7 0.00000000     20  0.37500 0.58190 0.043775
```

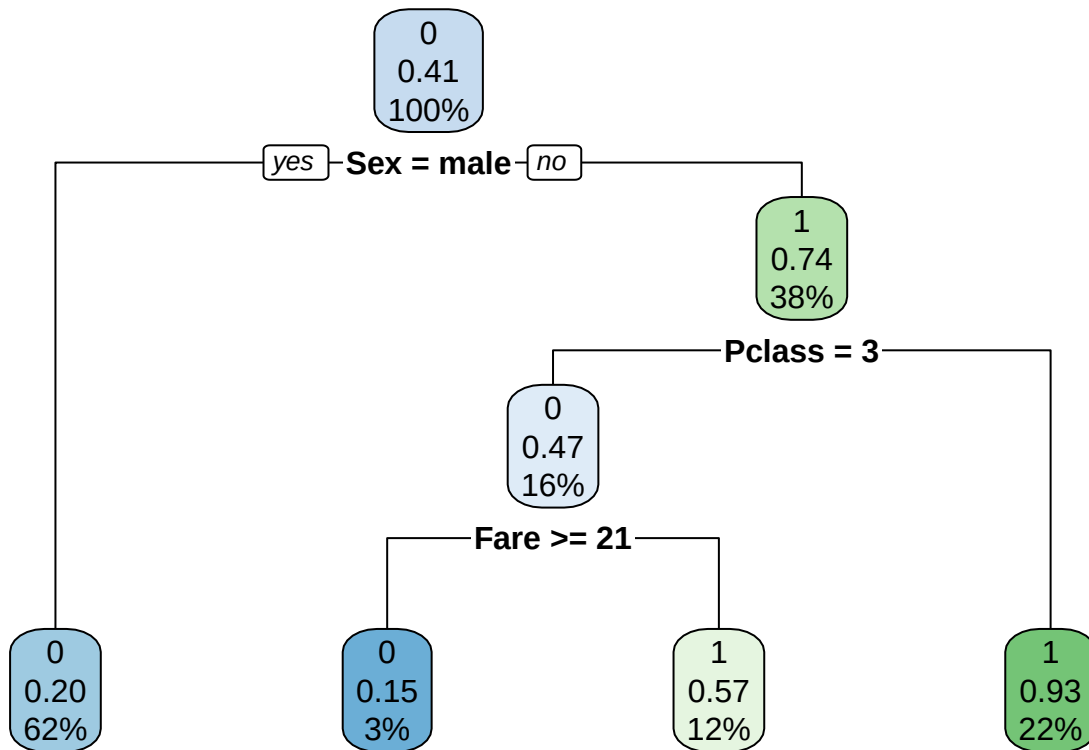
Access the best cp

```
best_cp <- full_tree$cptable[which.min(full_tree$cptable[, "xerror"]), "CP"]
best_cp
```

```
## [1] 0.01293103
```

Plot the best tree

```
pruned_tree <- prune(  
  full_tree,  
  cp=best_cp  
)  
  
rpart.plot(pruned_tree)
```



Prediction

```
pred_pruned <- predict(  
  pruned_tree,  
  test.data,  
  type = "class"  
)
```

Confusion Matrix

```
confusionMatrix(  
  factor(pred_pruned),  
  factor(test.data$Survived),  
  positive = "1"  
)
```

```
## Confusion Matrix and Statistics
##
##           Reference
## Prediction 0  1
##           0 79 22
##           1  5 36
##
##           Accuracy : 0.8099
##           95% CI : (0.7355, 0.8708)
##       No Information Rate : 0.5915
##       P-Value [Acc > NIR] : 2.405e-08
##
##           Kappa : 0.5878
##
## Mcnemar's Test P-Value : 0.002076
##
##           Sensitivity : 0.6207
##           Specificity : 0.9405
##       Pos Pred Value : 0.8780
##       Neg Pred Value : 0.7822
##           Prevalence : 0.4085
##       Detection Rate : 0.2535
##       Detection Prevalence : 0.2887
##       Balanced Accuracy : 0.7806
##
##       'Positive' Class : 1
##
```

Result

```
#full tree
table(pred_full, test.data$Survived)

##
## pred_full  0  1
##           0 71 11
##           1 13 47

#pruned tree
table(pred_pruned, test.data$Survived)

##
## pred_pruned  0  1
##           0 79 22
##           1  5 36
```

The pruned model increases TN (79 vs. 71), meaning it is better at correctly identifying dead passengers.

The pruned model has a lower TP (36 vs. 47), meaning it predicts fewer survived passengers correctly than the full model.

There is a trade-off. Chose the model based on your research goal.